Kodigo
ACADEMIA DE TECNOLOGÍA CREATIVA

Bootcamp: **Full Stack Junior**

Modulo 4: Profesionalización Fronted con TypeScript y Next.js

Semana 14

Contenido

1. *Middlewares en Next.js para protección de rutas globales.*
2. *NextJS administración de cookies para autenticación.*
3. *Autenticación avanzada (Session vs JWT).*
4. *Estrategias de SEO: Metadatos dinámicos*

Contenido:

1. Middlewares en Next.js 16+

En **Next.js 16+**, un **Middleware** es una función especial que se ejecuta **antes de que una solicitud (request) llegue a su destino final** (ya sea una página, un Route Handler o una API).

Es como un **filtro inteligente** que intercepta la petición y decide qué hacer con ella:

- Permitirla
- Redirigirla
- Reescribirla
- Bloquearla
- Modificarla

¿Dónde se ejecuta?

El Middleware se ejecuta en el **Edge Runtime**, lo que significa que corre **cerca del usuario**, en la red global de servidores (CDN/Edge).

En términos simples:

Es código que se ejecuta *antes* de que la solicitud llegue al servidor principal, mejorando velocidad y seguridad.

Esto permite:

- Redirecciones instantáneas
- Validaciones sin cargar JS del cliente
- Mayor seguridad



Ilustración 1 - Flujo del ciclo de vida de una solicitud - Fuente: Autor de contenidos

1.1. Creación y Configuración: El archivo middleware.ts

Ubicación del archivo

El archivo debe llamarse exactamente: middleware.ts

Y debe estar en:

- La **raíz del proyecto**, o
- Dentro de la carpeta `src/`

Ejemplo de estructura:

```
/app
/middleware.ts
/next.config.ts
/package.json
```

Ilustración 2 –Estructura de proyecto Nextjs 16+ - Fuente: Autor de contenidos

Solo puede existir **un archivo middleware.ts por aplicación**, porque actúa como punto

central de interceptación.

1.2. Configuración básica en TypeScript



```
import { NextRequest, NextResponse } from "next/server";

export function middleware(request: NextRequest) {
  console.log("Middleware ejecutado para:", request.nextUrl.pathname);

  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*", "/profile/:path*"],
}
```

Ilustración 3 – Middleware inicial en Nextjs 16+ - Fuente: Autor de contenidos

- **NextRequest:** Representa la solicitud entrante.
- **NextResponse:** Permite modificar la respuesta.
- **NextResponse.next():** Continúa el flujo normal.
- **Matcher:** Define qué rutas deben pasar por el middleware.

1.3. Protección de Rutas Basada en Autenticación

Uno de los usos más comunes del Middleware es la protección de rutas privadas.

Lógica de negocio

1. Leer una cookie o token
2. Validar si existe
3. Permitir acceso o redirigir

Aspecto	Validación en Cliente (useEffect)	Middleware (Edge)
Velocidad	Más lenta (espera al JS)	Instantánea (Edge)
Seguridad	Media (bypassable)	Alta (nivel servidor)
UX	"Flash" de contenido privado	Redirección limpia

Tabla 1 - Comparación técnica - Fuente: Autor de Contenidos

Ejemplo en Nextjs 16+

```ts
import { NextRequest, NextResponse } from "next/server";

export function middleware(request: NextRequest) {
  const sessionToken = request.cookies.get("session-token");

  const isAuthPage = request.nextUrl.pathname.startsWith("/login");
  const isProtectedRoute = request.nextUrl.pathname.startsWith("/dashboard");

  // Usuario intenta acceder a ruta protegida sin sesión
  if (isProtectedRoute && !sessionToken) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  // Usuario autenticado intentando acceder a login
  if (isAuthPage && sessionToken) {
    return NextResponse.redirect(new URL("/dashboard", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*", "/login"],
};
```

Ilustración 4 – Implementación de Middleware Nextjs 16+ - Fuente: Autor de contenidos

La validación en Middleware es más segura porque ocurre antes de que el contenido privado sea enviado al navegador.

1.4. Redirecciones vs. Reescritura (Rewriting) de URLs

En Next.js, el Middleware puede alterar el destino de una solicitud usando:

- **NextResponse.redirect()**
- **NextResponse.rewrite()**

Redirect

Cambia la URL visible en el navegador.

```ts
return NextResponse.redirect(new URL("/maintenance", request.url));
```

Ilustración 5 – Redirect Middleware Nextjs 16+ - Fuente: Autor de contenidos

El usuario ve /maintenance en la barra de direcciones.

Rewrite

Mantiene la URL visible, pero cambia el contenido que se renderiza.



Ilustración 6 – Rewrite Middleware Nextjs 16+ - Fuente: Autor de contenidos

El usuario sigue viendo `/home`, pero recibe el contenido de `/beta-version`.

Casos de uso

Usar Redirect cuando:

- Usuario no autenticado
- Modo mantenimiento
- Cambio permanente de URL (SEO)

Usar Rewrite cuando:

- A/B testing
- Feature flags
- Versiones beta ocultas
- Internacionalización

2. Administración de Cookies en Next.js 16+

Cuando hablamos de **autenticación**, necesitamos almacenar un **token** que identifique al usuario después del login.

En el navegador tenemos dos opciones comunes:

- **LocalStorage**
- **Cookies**

Diferencia clave

LocalStorage

- Accesible completamente desde JavaScript.
- Fácil de usar.
- Vulnerable a ataques **XSS (Cross-Site Scripting)**.
- No se envía automáticamente en cada request.

Cookies

- Se envían automáticamente en cada request HTTP.
- Pueden configurarse como **httpOnly** (no accesibles desde JS).
- Soportan configuraciones avanzadas de seguridad.
- Son el estándar de la industria para autenticación.

2.1. ¿Por qué NO usar LocalStorage para tokens?

Si un atacante logra inyectar un script (XSS), puede hacer:

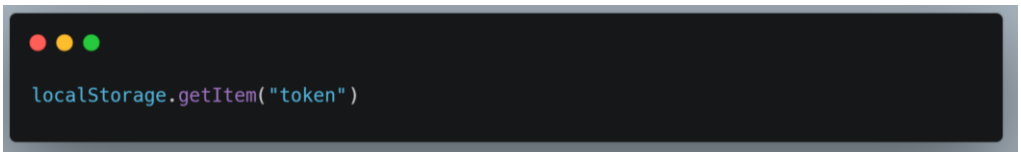


Ilustración 7 – LocalStorage - Fuente: Autor de contenidos

Y robar el token fácilmente. En cambio, una cookie con httpOnly: true:

- No puede leerse desde JavaScript.
- Solo el servidor puede acceder a ella.

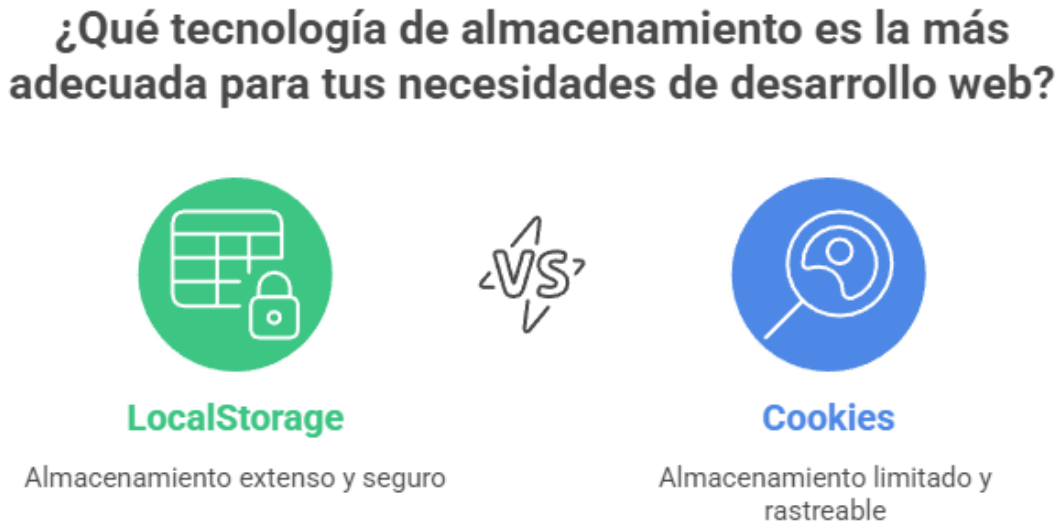


Ilustración 8 – LocalStorage vs Cookies - Fuente: Autor de contenidos

2.2. Configuración Segura (El Estándar de la Industria)

Cuando creamos una cookie para autenticación, debemos configurarla correctamente.

Estas son las tres propiedades más importantes:

Atributo	Propósito	Por qué es vital en Auth

httpOnly	Impide acceso vía JavaScript.	Evita el robo de tokens por scripts maliciosos (XSS).
Secure	Solo se envía por HTTPS.	Protege el token de ser interceptado en redes no seguras.
SameSite	Controla el envío en peticiones cross-site.	Previene ataques de tipo CSRF.

Tabla 2 –Propiedades de las cookies - Fuente: Autor de Contenidos

httpOnly

La cookie no puede leerse con document.cookie.
Protección contra **XSS**.

Secure

La cookie solo viaja si la conexión es HTTPS.
Protección contra interceptación en redes públicas.

SameSite

- **Lax:** Permite algunos envíos cross-site (más flexible).
- **Strict:** Solo se envía si el usuario navega dentro del mismo sitio.

2.3. Configuración Segura (El Estándar de la Industria)

En Next.js 16+ (App Router), las cookies se manejan principalmente en el servidor, usando:

```
import { cookies } from "next/headers";
```

Ilustración 9 – Importación API de cookies - Fuente: Autor de contenidos

Esto funciona en:

- Server Components
- Server Actions
- Route Handlers

Ejemplo: Cookie en un Login (Server Action)

```server.ts
"use server";

import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function loginAction(formData: FormData) {
  const email = formData.get("email");
  const password = formData.get("password");

  // Simulación de validación
  if (email === "admin@test.com" && password === "123456") {
    cookies().set("token", "secure-session-token", {
      httpOnly: true,
      secure: true,
      sameSite: "lax",
      path: "/",
    });

    redirect("/dashboard");
  }

  throw new Error("Credenciales inválidas");
}
```

Ilustración 10 – Implementación de login con cookies - Fuente: Autor de contenidos

2.4. Lectura y Validación de Tokens

Para proteger rutas o personalizar contenido, debemos leer la cookie.

```layout.tsx
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const token = cookies().get("token")?.value;

  if (!token) {
    redirect("/login");
  }

  return <>{children}</>;
}
```

Ilustración 11 – Ejemplo: Leer Cookie en un Layout - Fuente: Autor de contenidos

2.5. Borrado de Cookies (Logout)

Cerrar sesión correctamente significa:

- No basta con limpiar estado de React.
- No basta con borrar contexto.
- Debes eliminar la cookie del navegador.



```
"use server";

import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function logoutAction() {
  cookies().delete("token");

  redirect("/login");
}
```

Ilustración 12 – Ejemplo: Logout con Server Action - Fuente: Autor de contenidos

Cuando se elimina la cookie:

- El navegador deja de enviarla.
- El servidor ya no reconoce la sesión.
- El usuario queda efectivamente deslogueado.

3. Autenticación avanzada (Session vs JWT)

Cuando un usuario inicia sesión, el servidor necesita responder una pregunta crítica en cada request:

“¿Quién eres tú y cómo lo sé?”

HTTP es un protocolo **stateless** (sin estado).
Eso significa que **cada petición es independiente** y el servidor no recuerda nada automáticamente.

Stateful vs Stateless

Stateful (Con Estado)

El servidor **guarda información del usuario** en memoria o base de datos.

Ejemplo:

- El servidor crea una sesión.
- Guarda datos en una tabla de sesiones.
- Asocia al usuario con un sessionId.

Stateless (Sin Estado)

El servidor **no guarda información del usuario**.
El cliente envía toda la información necesaria en cada request (generalmente en un token firmado).

3.1. Sesiones Stateful

Flujo de una Sesión

1. Usuario hace login.
2. El servidor valida credenciales.
3. El servidor crea una sesión en DB o memoria.
4. Se genera un sessionId.
5. El cliente recibe una cookie con ese ID.

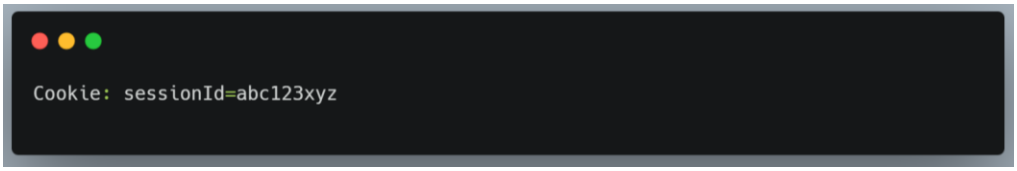


Ilustración 13 – Ejemplo conceptual - Fuente: Autor de contenidos

El servidor busca ese ID en cada request para identificar al usuario.

Ventaja	Desventaja
Revocación inmediata (puedes cerrar sesión desde el server).	Dificultad para escalar (Sticky sessions o Redis).
Menor carga de datos en el cliente.	Dependencia total de una base de datos de sesiones.

Tabla 3 – Pros y Contras sesiones stateful - Fuente: Autor de Contenidos

3.2. Sesiones Stateless

JWT significa **JSON Web Token**.

Un JWT tiene tres partes separadas por puntos:

- **Header:** Algoritmo y tipo (HS256, RS256)
- **Payload:** Datos del usuario (claims como `id`, `email`, exp)
- **Signature:** Firma criptográfica que valida integridad



Ilustración 14 – Ejemplo visual JWT - Fuente: Autor de contenidos

El servidor no guarda nada, solo verifica la firma con su clave secreta.

Ejemplo TypeScript (Decodificar sin verificar)



```typescript
import jwtDecode from "jwt-decode";

interface TokenPayload {
  id: string;
  email: string;
  exp: number;
}

const token = "tu.jwt.aqui";

const decoded = jwtDecode<TokenPayload>(token);
console.log(decoded.email);
```

Ilustración 15 – Decodificación JWT - Fuente: Autor de contenidos

3.3. Refresh tokens

Problema: ¿Qué pasa si roban un JWT?

Si el Access Token tiene larga duración:

- El atacante puede usarlo hasta que expire.

Solución Profesional

- **Access Token:** Vida corta (5–15 minutos)
- **Refresh Token:** Vida larga (7–30 días)
- Guardado en cookie httpOnly.

Flujo de Trabajo

1. El Access Token expira.
2. El cliente recibe 401.
3. Se usa el Refresh Token para pedir uno nuevo.
4. El usuario no nota nada (Silent Refresh).

4. Estrategias de SEO y Metadatos Dinámicos en Next.js 16+

4.1. ¿Qué es el SEO técnico?

El **SEO técnico** es el conjunto de optimizaciones estructurales que permiten a los motores de búsqueda (como Google) **entender, rastrear e indexar correctamente un sitio web**.

No se trata solo de palabras clave. Se trata de:

- HTML bien estructurado
- Metadatos correctos
- Rendimiento
- Accesibilidad
- Indexabilidad

4.2. ¿Por qué Next.js lidera en SEO?

Next.js es potente en SEO porque genera HTML en el servidor usando:

- **SSR (Server-Side Rendering)**
- **SSG (Static Site Generation)**
- **RSC (React Server Components)**

Esto significa que cuando un bot de Google llega a tu sitio:

Ya recibe el contenido renderizado, no una página vacía esperando JavaScript.

En contraste, una SPA tradicional envía:

```
<div id="root"></div>
<script src="bundle.js"></script>
```

Ilustración 16 – SPA tradicional - Fuente: Autor de contenidos

4.3. Metadatos Estáticos vs Dinámicos

- **Estáticos:** No cambian (Home, Contacto).
- **Dinámicos:** Cambian según datos (Producto por ID, Post por slug).

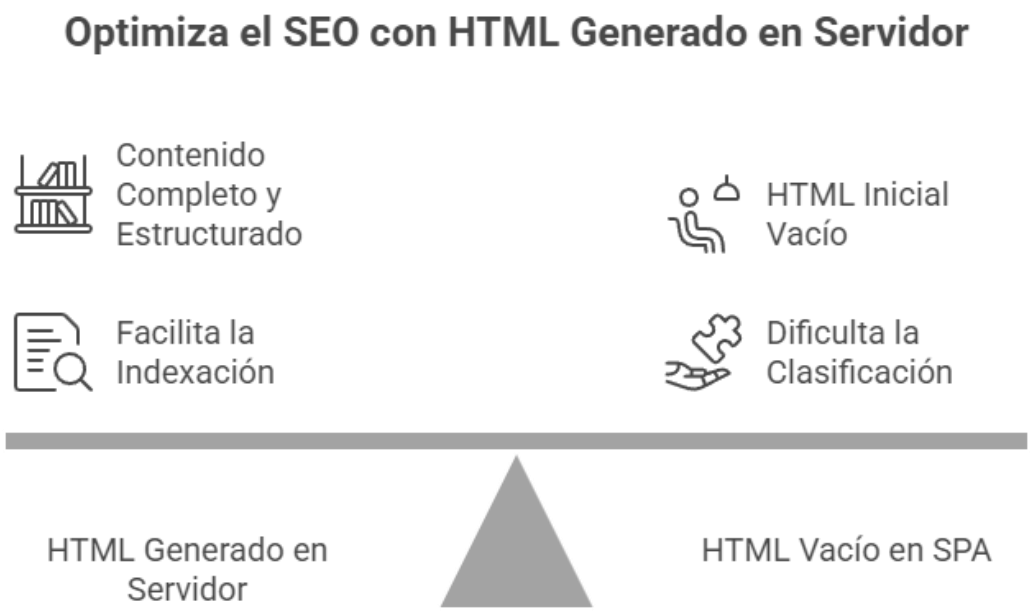


Ilustración 17 – Metadatos Estáticos vs Dinámicos - Fuente: Autor de contenidos

Google hoy en día usa un proceso llamado **Rendering en dos fases**: primero indexa el HTML crudo, luego renderiza JavaScript en una segunda etapa. Si tu SEO depende de JS, puedes perder posicionamiento.

4.4. Api de metadata: Statico vs dinamico

Next.js 16+ (App Router) ofrece una API moderna y tipada para manejar metadatos.

Método	Caso de Uso	Ubicación
Object Metadata	Páginas estáticas (Home, Contacto).	Exportación constante en layout.ts o page.ts.
generateMetadata	Páginas dinámicas (Producto por ID, Post por Slug).	Función asíncrona dentro de page.ts.

Tabla 4 – Comparación Oficial metadata - Fuente: Autor de Contenidos

Metadatos Estáticos

```ts
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "Inicio | Mi Tienda",
  description: "La mejor tienda online de tecnología moderna",
};
```

Ilustración 18 – Metadatos Estáticos - Fuente: Autor de contenidos

Metadatos Dinámicos con generateMetadata

Ejemplo: Página de producto dinámica.



```tsx
import type { Metadata, ResolvingMetadata } from "next";

interface Product {
  id: string;
  title: string;
  description: string;
}

interface PageProps {
  params: { id: string };
}

async function getProduct(id: string): Promise<Product> {
  const res = await fetch(`https://api.example.com/products/${id}`);
  return res.json();
}

export async function generateMetadata(
  { params }: PageProps,
  parent: ResolvingMetadata
): Promise<Metadata> {
  const product = await getProduct(params.id);

  return {
    title: product.title,
    description: product.description,
  };
}
```

Ilustración 19– Metadatos Dinámicos - Fuente: Autor de contenidos

Recursos de Apoyo



Guia
Guía de implementación de autenticación.

Guia
Guía de configuración de cookies.

Video
Tutorial de autenticación segura con NextJS